

Datadog Live Debugger onboarding audit

Beginner experience and professional QA for Live Debugger and Source Code Integration

Work in progress | Updated 8 October 2026 at 03:02 UTC

Executive summary

The public onboarding path contains a reproducible Python command defect and a desktop documentation layout defect. Authenticated product testing is still in progress; the results below cover only the evidence established so far.

- **Fix the Python quickstart first.** The documented module command includes a .py suffix and fails for an ordinary Python script. Both corrected forms pass the isolated local check.
- **Make navigation labels readable.** At the observed desktop viewport, the documentation contents rail breaks several section names mid-word.
- **Explain the first irreversible decision.** Region selection is clearly marked permanent, but a complete beginner receives little in-place help choosing one.

Severity index below: HIGH 0 · MEDIUM 1 · LOW 2. Click a finding ID to jump to its detail page.

[Open the High severity only view](#)

Decision boundaries

A documented prerequisite is not proof that it worked. This report distinguishes directly observed behavior, local reproduction, UX recommendations, and tests not run. Real capture, exact source revision, authorization boundaries, expiry, and cleanup require separate evidence.

Record	Classification	Observed impact
DOC01	MEDIUM · bug	Python portion fails the isolated module invocation check
DOC02	LOW · bug	Navigation labels break mid-word at desktop width
UX01	LOW · UX	Permanent region choice precedes explanatory context

Highest value retests

After the command fix, run one clean Python quickstart through first real variable capture. Then verify the exact deployed commit in source view, an A-only repository grant, stop/expiry behavior, and safe recovery from missing prerequisites.

Scope and interpretation

The requested outcome is a real end-to-end debugging journey: create a fresh account, find the feature, prepare a simple application, observe runtime values, connect the matching source, and stop collection cleanly.

Two distinct review tracks

Beginner track. Assess what the page itself explains to someone unfamiliar with Datadog and APM. Record the first path, unclear terms, missing prerequisites, progress feedback, and recovery steps. Confusion is a UX assessment unless a specific promised behavior fails.

Professional QA track. Verify identity, configuration, deployed source revision, runtime state, error handling, permissions, lifecycle, and data presentation. A green badge or successful save is not evidence of a real event or correct source.

Evidence rules

- Actual browser screenshots and continuous screen capture support UI observations. Red outlines are editable report overlays; screenshot content is unchanged.
- A local Python process check isolates the quickstart syntax. It does not validate the ddtrace-run wrapper, Agent, log ingestion, or a long-running instrumented application.
- Only observed states are called passed or failed. Blocked and not-run cases remain explicit, including parts of otherwise useful partial tests.
- Dates and times are UTC. Screenshot-file timestamps and recording offsets are identified separately; approximate offsets are not capture clocks.

Severity definitions

Severity	Meaning
HIGH	Core task blocked or significant incorrect behavior with no reasonable workaround
MEDIUM	Material task failure or misleading behavior with a practical workaround
LOW	Readability, discoverability, or localized friction without demonstrated task failure

Environment

Cloud desktop browser; approximately 1180 × 757 page viewport at default zoom. Documentation evidence images are 1165 × 747 pixels. Local reproduction uses Python 3.12.14. A dedicated synthetic sample contains no real customer data. Final account/runtime versions and coverage will be reconciled at audit close.

The beginner journey

Stage	What a beginner needs to understand	Observation
Choose region	Where the account lives and whether the decision can change	Permanent-choice warning is visible; choosing criteria are not explained in the captured form
Find the feature	How APM, Live Debugger, sessions, and logpoints relate	Authenticated path still under review
Prepare an app	Which commands run on the host, in the app, and in the Agent	Docs surface several independent prerequisites; Python launch syntax is faulty
Get a first result	Difference between service discovery, active logpoint, and captured data	Runtime path still under review
Connect source	Difference between repository access and exact deployed-commit mapping	Integration path still under review
Stop safely	How expiry and disable affect collection and old results	Lifecycle path still under review

Make the first success concrete

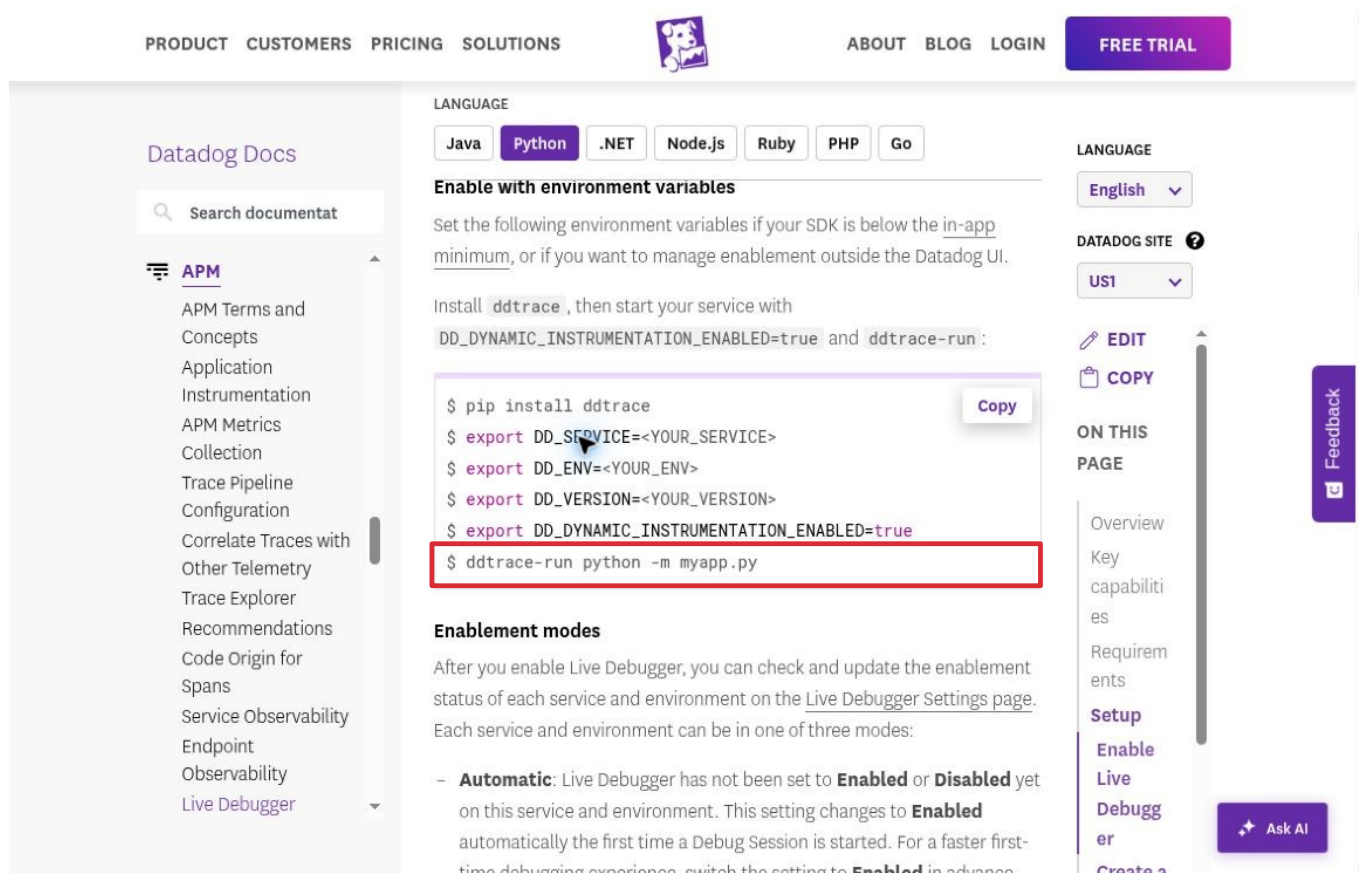
Show a short path from one running test service to one visible captured value. State which setup checks are complete and which still need action. Keep the manual path visible when source integration is optional, then explain what source linking adds.

Do not collapse distinct states

A beginner should be able to tell whether an app is unknown, not instrumented, waiting for activation, receiving no matching traffic, or capturing data that is hidden by filters. These are test questions, not unverified defects.

Python quickstart contains an invalid module command

The launch example uses `python -m myapp.py`. Python's module switch expects a module name rather than a script filename.



E04 · Official Live Debugger documentation, Python tab · Screenshot file 02:20:19 UTC. Red outline identifies the launch command.

Video evidence 00:58 to 01:20

Reproduce Open Live Debugger docs, select Python, then Enable with environment variables. Follow the final launch line with an ordinary `myapp.py` script.

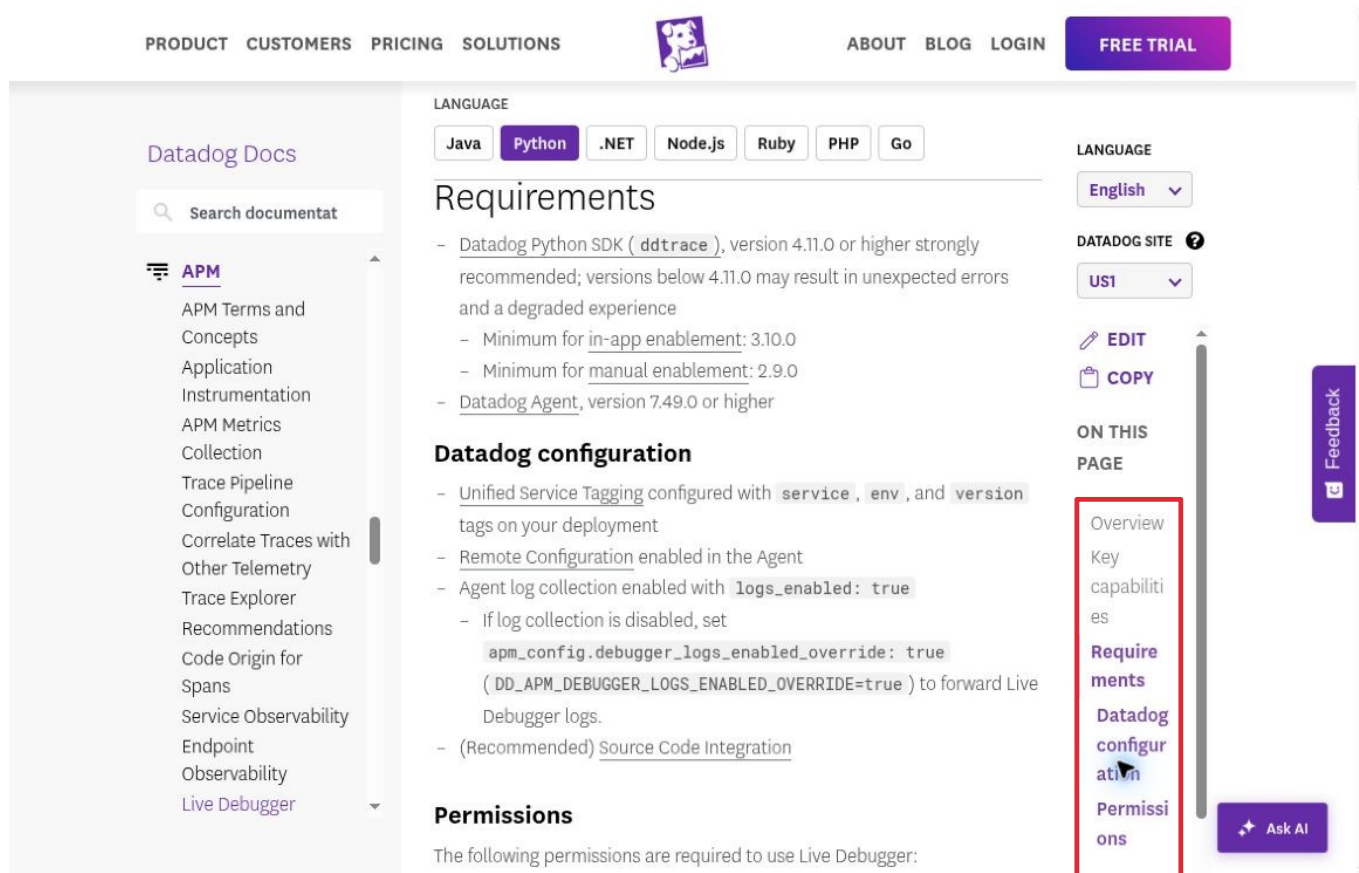
Expected A copied quickstart command starts the intended Python application without a module-resolution error.

Actual The Python portion exits 1 for a minimal script. Import-time code runs before the error, so this check does not imply that no application code executes.

Fix and retest Use `ddtrace-run python myapp.py` for a script, or `ddtrace-run python -m myapp` for a module. Add a quickstart smoke test that asserts successful process startup.

Documentation navigation breaks words at desktop width

The right-hand contents rail wraps ordinary section names in the middle of words, making the onboarding outline harder to scan.



E03 · Requirements page · Screenshot file 02:19:51 UTC · 1165 × 747 captured page image; default browser zoom.

Video evidence 00:34 to 00:58

Reproduce Open the Python Live Debugger documentation at the observed desktop width and scroll to Requirements. Inspect the right-hand contents rail.

Expected Section names remain readable, either wrapping at word boundaries or collapsing into a usable navigation control.

Actual Labels such as Requirements, Permissions, and Datadog configuration split mid-word. The defect is visible in two separate scroll positions.

Fix and retest Reserve sufficient rail width or collapse the rail earlier. Retest default desktop widths and 125–150% zoom, including long translated labels.

A permanent region choice needs decision support

The signup form clearly warns that the selected region cannot be changed later. The captured form does not explain what should drive that choice.

Datadog named a Leader in the Gartner® Magic Quadrant™ for Observability Platforms

PRODUCT CUSTOMERS

Start Your Free Trial Now

No credit card required

Try it free for 14 days and monitor as many servers as you like.

Select a Region *

United Kingdom (UK1)

Important: This can't be changed later

Business Email *

Full Name *

Company *

Password *

Use at least 8 characters containing at least 1 number and 1 lowercase letter

Phone

*Required Fields. By signing up, you agree to the [Master Subscription Agreement](#), [Privacy Policy](#), and [Cookie Policy](#)

Sign Up With Google Sign Up

UX OBSERVATION | Permanent region choice appears during trial signup
The warning is clear; explaining how to choose could help a first-time user.

E08 · Actual edited-video frame at 00:20, sourced from raw capture 00:50, about 02:15:17 UTC. All identity fields are blank.

Video evidence 00:20 to 00:34

Reproduce Start a fresh trial and inspect Select a Region before submitting the form.

Recommended experience Give a short explanation or linked chooser covering account location and relevant selection criteria before the irreversible step.

Assessment The permanent-choice warning is explicit and useful. Additional decision support would reduce guesswork for a complete beginner.

Boundary This is an onboarding recommendation. No wrong-region account, data loss, or failed submission was demonstrated.

Professional QA coverage

QA_MATRIX.md contains 54 baseline cases and nine supplementary edge cases, all planned. They describe intended coverage, not tests executed. Each case remains NOT RUN until direct evidence establishes a result; the confirmed documentation findings and local tests are reported separately.

Area	Evidence established	Still needs end to end validation
Documentation	Invalid Python module invocation; cramped contents rail	Fresh instrumented quickstart after correction
Runtime fixture	8 passing local tests; 1 expected failure for the planted bug	Agent/SDK integration, real service discovery and captured locals
Source identity	Exact source and commit checks specified	Deployed SHA, path/line, branch drift and mixed versions
Access boundaries	A-only grant and never-granted B test specified	Repository scope, stale grants and independent roles
Session lifecycle	Expiry and cleanup test oracles specified	Actual expiry, stop, re-enable and event timestamps
Robustness	Negative cases prioritized	Malformed input, stale tabs, duplicate submission, failure recovery

Permission tests need precise interpretation

An organization integration can legitimately share access according to its grant model. Do not call lack of a per-user GitHub check a vulnerability without verifying that model. Old source may remain cached for up to seven days under the documented contract; revocation tests must distinguish cached content from a newly fetched, never-seen revision.

High risk tests before confidence in release readiness

- Capture actual quantity 3 locals at pricing.py line 17 and match the correct service, environment, and deployed revision.
- Confirm repo A works while never-granted private repo B cannot return source through the tested integration.
- Observe capture before and after real expiration, distinguishing late ingestion from late execution.
- Reload after stopping and verify every created test session/logpoint is inactive.

Reproduction and fixture oracle

Command isolation

Independent reproduction: 02:31:35 UTC on 8 October 2026. The file myapp.py contains one harmless print statement; no Datadog SDK or telemetry is involved.

```
myapp.py:
print("INERT_FIXTURE_EXECUTED")

$ python -m myapp.py
INERT_FIXTURE_EXECUTED
ModuleNotFoundError: __path__ attribute not found on myapp
Exit: 1

$ python myapp.py
INERT_FIXTURE_EXECUTED
Exit: 0

$ python -m myapp
INERT_FIXTURE_EXECUTED
Exit: 0
```

The error is abbreviated for readability. Python also suggests using myapp instead of myapp.py as the module name. The working alternatives establish the module-versus-filename distinction; neither has been claimed as an instrumented Datadog success.

Known sample behavior

The synthetic quote service intentionally uses quantity > 3 instead of quantity >= 3. It is a target bug in the demo application, not a Datadog defect. At pricing.py line 17 all calculated locals are available for the intended line logpoint.

Quantity	Subtotal	Discount eligible	Discount	Actual total
2	2400	false	0	2400
3	3600	false	0	3600
4	4800	true	480	4320

All monetary values are integer cents. The intended quantity 3 total is 3240. A valid debugger result should expose the actual 3600 and the false eligibility, making the boundary error explainable.

Local test rerun at 02:36 UTC: nine tests completed, eight passed and one intentional expected failure. HTTP tests bind to loopback and clean up their server.

Evidence and reference index

ID	Evidence	UTC time or recording location
E08	Blank Datadog trial form video frame	Edited video 00:20; raw 00:50; approximately 02:15:17 UTC
E03	Python requirements and navigation	File timestamp 02:19:51; initial video approximately 05:15–05:45
E04	Python environment-variable launch example	File timestamp 02:20:19; initial video approximately 05:45–06:20
R01	Independent Python invocation reproduction	02:31:35; Python 3.12.14; bad form exits 1, alternatives exit 0
R02	Local fixture test rerun	02:36; 8 passes, 1 intentional expected failure

The 80-second edited excerpt covers public signup and documentation only. Its chapters are 00:20 region choice, 00:34 contents-rail wrapping, and 00:58 invalid Python command. Authenticated account-creation footage is not claimed by this excerpt.

The initial continuous recording began at 02:14:27 UTC and ended at 02:21:01 UTC. It is 393.7 seconds at 1364 × 1024 pixels, 20 fps. Edited-video offsets will differ from raw recording offsets. E03 and E04 are direct page captures; E08 is an extracted video frame. Published evidence excludes browser chrome.

Primary product references

[Live Debugger setup and session behavior](#)

[Python source mapping](#)

[Source Code Management Providers and caching](#)

[GitHub integration](#)

[Live Debugger expression language](#)

References inform expected behavior and test design. They are not substitutes for observed product results. All current findings are limited to the captured site, viewport, and isolated runtime check.

Planned scenario coverage

The baseline matrix contains 54 cases. The following groups explain the intended coverage in plain language; their outcomes remain NOT RUN until evidence is recorded. P0 is test priority, not defect severity.

Scenario group	Baseline IDs	Result to establish
Fresh account to first capture	B01–B08, F01	A beginner can reach a real captured value and explain the sample bug
Missing prerequisites	B02, B03, B06	Missing APM, Remote Configuration, runtime compatibility, or roles have distinct recovery guidance
Traces but no debugger service	B02, B05, B07	An APM-visible service that is absent from debugger selection has an actionable explanation
Repository A allowed and B denied	S01–S04, S11	A works, never-granted private B is denied, and recovery keeps the intended scope
Revoked source and caching	S10, S15	Separate cached old contents from newly fetched never-seen source and verify the actual grant model
Runtime commit and moving files	S02, S05–S09	Branch drift, missing commit, renamed paths, and mixed versions cannot silently show wrong code
Invalid targets and expressions	F04–F07	Bad lines, unknown variables, invalid syntax, and unsupported context fail usefully
Never true and erroring conditions	F02–F05	No matching traffic is distinguishable from expression errors and inactive installation
Duplicate and stale actions	F08–F10, F14	Double clicks, old tabs, back/refresh, and canceled drafts do not hide unintended changes
Expiry and restart	F11–F13, F18–F20	Capture stops at the effective lifecycle boundary and recovery uses the correct target
Permission loss	S12–S16	Lost read/write access and cross-org navigation respect effective authorization
Variable safety and complexity	F16, F21–F23	Redaction, truncation, types, nesting, and rate limits are represented honestly
Cleanup and minimal reproduction	C01–C03	Every test entity is reconciled and observed failures have a fresh-state positive control

The full executable preconditions, steps, expected outcomes, evidence, and cleanup requirements remain in QA_MATRIX.md. Runtime and access tests require the appropriate synthetic deployment and isolated permission setup.

Additional edge case plan

These nine supplementary cases preserve the original 54-case matrix. They add specific concurrency and deployment challenges; none is an observed defect or executed result.

ID	Scenario	Evidence required before a result
X01	Mixed-commit rolling deployment	Two active replicas of the same service carry different commits and numeric markers. Each capture resolves to its executing replica's source.
X02	Same service names across environments and forks	Use distinct environment, owner/repo, and numeric markers. Filters and direct links must never silently attach a capture to the wrong target.
X03	Never-executed function versus uninstalled probe	Compare a healthy installed location with no executions against a failed installation. UI and runtime evidence distinguish these cases.
X04	Delete or expiry versus late snapshots	Record execution, expiry/delete, and ingest times. Buffered old snapshots cannot be mistaken for new capture after the effective stop.
X05	A selected line moves during deployment	Shift the statement in a new commit and roll forward. Source location and installed definition remain correctly attributed or explicitly require a new target.
X06	Asynchronous concurrency correlation	Run bounded overlapping synthetic requests with unique numeric IDs. Captured locals, request/trace context, and selected source belong to the same execution.
X07	Cyclic Unicode values and nested sensitive fields	Capture small cyclic structures, Unicode keys/values, and nested dummy sensitive identifiers under unchanged protection. Inspect safe truncation and masking.
X08	Repository recovery without reinstall	Remove and restore an approved test-only prerequisite or grant. Correct source becomes available again without unexplained full reinstall or expanded scope.
X09	Same-location sessions with different conditions	Run two explicit sessions at the same line with disjoint predicates. Captures, edits, stop, and expiry remain associated with the correct session.

Each case requires a baseline positive control, exact identities, UTC execution/capture timestamps, a reload or independent state check, and cleanup. Any missing prerequisite is a blocked test, not a product failure.

Beginner glossary

These terms describe the debugging workflow. A visible setup check is not the same as a captured event.

Term	Plain language meaning
APM	Application Performance Monitoring. Traces show how requests move through application operations.
Agent	A process near the application that receives and forwards supported telemetry and configuration.
SDK or tracer	A language-specific library in the application. Python uses ddtrace.
Trace and span	A trace follows a request or task; a span represents one operation within it.
Service	The named application or component being monitored. Names alone do not establish environment or source identity.
Environment and version	Tags identifying where the application runs, such as staging, and which release it runs.
Remote Configuration	A channel for supported configuration changes to reach a running Agent or SDK without a new deployment.
Debug session	A bounded investigation containing debugger configuration and related results. Check its actual expiry and active state.
Logpoint	A location and rule that collect a message or values without editing the deployed source for each observation.
Snapshot or captured locals	Values observed at a specific runtime execution. Availability, redaction, sampling, and truncation affect what appears.
Source Code Integration	Repository access plus mapping that connects runtime evidence to source files. A connected account alone is insufficient.
Commit SHA	A Git revision identifier. The source shown for a capture should match the deployed commit, not merely the latest branch.
Redaction	Protection that removes or masks selected data. Missing, unavailable, and redacted values mean different things.